

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD DE INGENIERÍA EN CIENCIAS Y SISTEMAS
Laboratorio Organización de lenguajes y compiladores 1
SECCIÓN C
PRIMER SEMESTRE 2026



Manual tecnico GoLite

Nombre :Adilzon Alfredo Velásquez Hernández
Carné :201908076

Guatemala, Viernes 12 de Junio del 2026

Introducción

Este manual técnico describe la arquitectura, diseño e implementación del intérprete del lenguaje GoLite, desarrollado como proyecto para el curso de Organización de Lenguajes y Compiladores 1 en la Universidad San Carlos de Guatemala.

El proyecto consiste en un intérprete completo que incluye análisis léxico, análisis sintáctico, construcción del AST (Árbol de Sintaxis Abstracta) y un intérprete que recorre el AST para ejecutar el código fuente escrito en GoLite.

2. Requisitos del Sistema

Componente	Versión	Descripción
Java JDK	21 o superior	Lenguaje base del proyecto
Maven	3.8+	Gestor de dependencias y construcción
JFLEX	1.9.1	Generación del analizador léxico (vía plugin Maven)
CUP	11b-20160615-3	Generación del analizador sintáctico (vía plugin Maven)
RSyntaxTextArea	3.3.4	Editor con resaltado de sintaxis en la GUI

3. Estructura del Proyecto

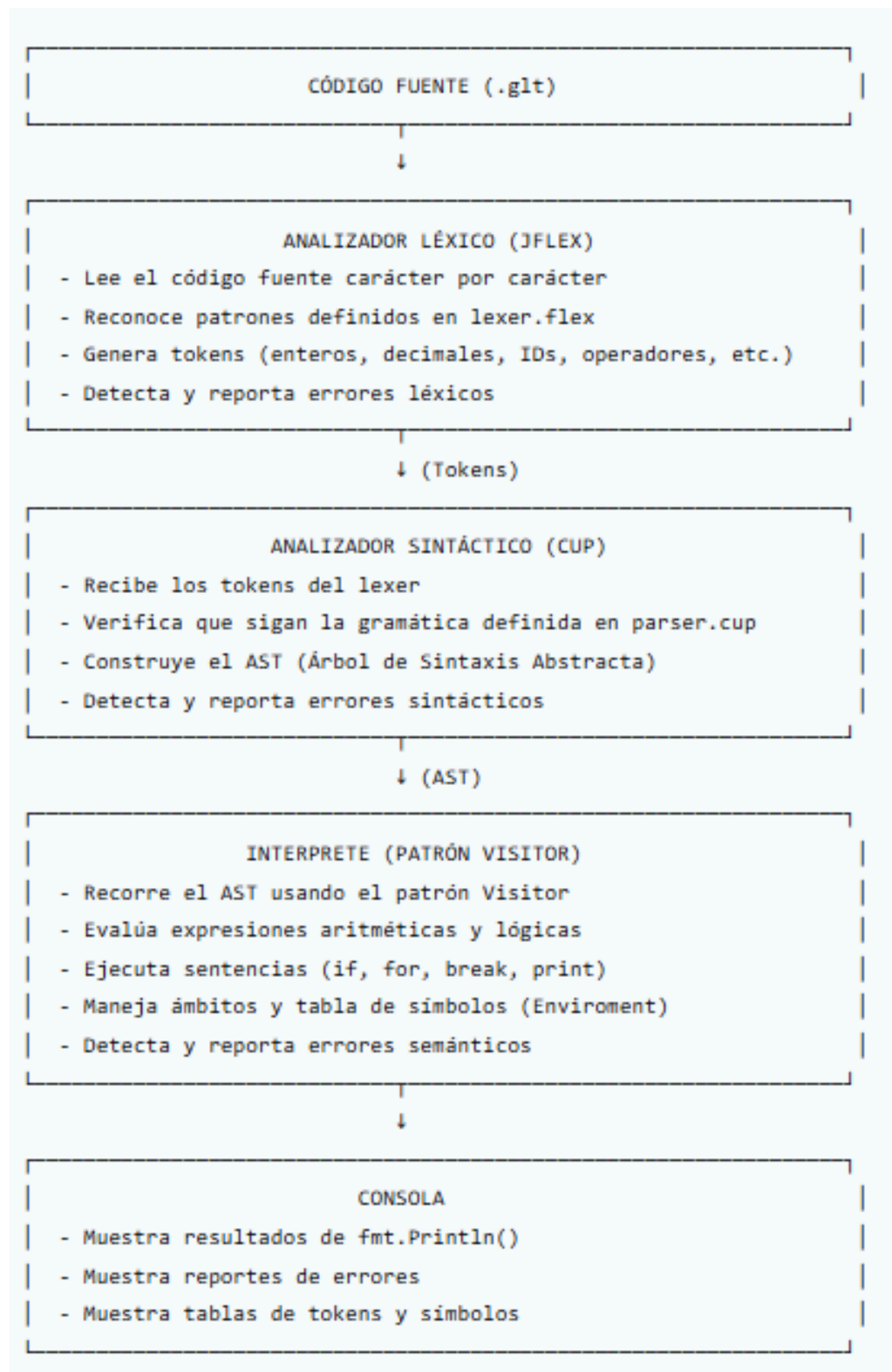
clase06/

```
|— pom.xml          # Configuración Maven
|— README.md        # Descripción del proyecto
|— Manual_Tecnico.pdf      # Este documento
|— Manual_Usuario.pdf     # Manual de usuario
|
|_ src/
    |_ main/
        |_ java/olc1/golite/
            | |— Main.java      # Punto de entrada
            | |
            | |— ast/          # Nodos del AST
            | |   |_ ASTNode.java  # Interfaz base
            | |   |_ exp/        # Nodos de expresiones
            | |       |_ Add.java   # Suma
            | |       |_ Sub.java   # Resta
            | |       |_ Mul.java   # Multiplicación
            | |       |_ Div.java   # División
            | |       |_ Lt.java    # Menor que
            | |       |_ Negate.java # Negación unaria
            | |       |_ Integers.java # Literal entero
            | |       |_ Decimal.java # Literal decimal
            | |       |_ BoolLiteral.java # Literal booleano
            | |       |_ StringLiteral.java # Literal cadena
```

- | | | └─ VarRef.java # Referencia a variable
- | | └─ stm/ # Nodos de sentencias
- | | └─ Assign.java # Asignación
- | | └─ Allocate.java # Reasignación
- | | └─ Imprimir.java # Print
- | | └─ IfNode.java # If-else
- | | └─ ElifNode.java # Else if
- | | └─ ElifNodes.java # Lista de elif
- | | └─ WhileFor.java # Bucle for
- | | └─ Breakstm.java # Break
- | | └─ Statments.java # Bloque de sentencias
- | |
- | └─ reports/ # Manejo de errores
- | | └─ GoliteError.java # Estructura de error
- | |
- | └─ views/ # Interfaz gráfica
- | | └─ GoliteFrame.java # Ventana principal
- | | └─ EditorPanel.java # Panel del editor
- | | └─ GoliteMenuBar.java # Barra de menú
- | | └─ MainPanel.java # Panel principal (split)
- | |
- | └─ visitor/ # Patrón Visitor
- | | └─ Visitor.java # Interfaz Visitor
- | | └─ interpreter/ # Visitor intérprete
- | | └─ InterpreterVisitor.java

```
|      |— Enviroment.java
|      |— transfer/
|      |  └─ BreakException.java
|      └─ value/
|          |— ValueWrapper.java
|          |— IntValue.java
|          |— DecimalValue.java
|          |— BoolValue.java
|          |— StringValue.java
|          └─ VoidValue.java
|
|— jflex/
|  └─ lexer.flex      # Especificación del lexer
|
└─ cup/
    └─ parser.cup      # Especificación del parser
```

4. Diagrama de Arquitectura



Generación de Analizadores

5.1 JFLEX (Analizador Léxico)

Archivo: `src/main/jflex/lexer.flex`

Para esta parte, utilicé la librería JFLEX porque es la herramienta estándar para generar analizadores léxicos en Java. Decidí usarla en lugar de implementar un lexer manualmente porque:

- Reduce drásticamente el código manual
- Es más fácil de mantener y modificar
- Se integra bien con Maven mediante el plugin `jflex-maven-plugin`

Patrones definidos en el lexer:

Categoría	Patrón	Token generado	Ejemplo
Enteros	<code>[0-9]+</code>	<code>integer</code>	123, 45
Decimales	<code>[0-9]+\.[0-9]+</code>	<code>decimal</code>	3.14, 2.5
Identificadores	<code>[a-zA-Z_][a-zA-Z0-9_]*</code>	<code>id</code>	x, contador, _temp
Cadenas	<code>"([^\\"] \\.)*"</code>	<code>string</code>	"Hola mundo"
Operador suma	<code>"+"</code>	<code>plus</code>	+
Operador resta	<code>"-"</code>	<code>minus</code>	-
Operador multiplicación	<code>"*"</code>	<code>times</code>	*
Operador división	<code>"/"</code>	<code>slash</code>	/

Operador menor que	"<"	lt	<
Asignación	assign	:=	
Reasignación	"="	allocate	=
Punto y coma	";"	scol	;
Paréntesis izquierdo	"("	lparen	(
Paréntesis derecho	")"	rparen	)
Llave izquierda	"{"	lbrace	{
Llave derecha	"}"	rbrace	}
Palabra reservada	"print"	imprimir	print
print			
Palabra reservada	"true"	kwTrue	true
true			
Palabra reservada	"false"	kwFalse	false
false			
Palabra reservada if	"if"	kwIf	if
Palabra reservada	"else"	kwElse	else
else			
Palabra reservada	"for"	kwFor	for
for			

Palabra reservada	<code>"break"</code>	<code>kwBreak</code>	<code>break</code>
	<code>break</code>		

Manejo de errores léxicos:

Cuando el lexer encuentra un carácter no reconocido, crea un objeto `GoLiteError` con tipo "Léxico", una descripción y la posición (línea, columna). Esto permite mostrar todos los errores al final del análisis.

5.2 CUP (Analizador Sintáctico)

Archivo: `src/main/cup/parser.cup`

Para el análisis sintáctico, utilicé CUP (Constructor of Useful Parsers) porque es la herramienta complementaria a JFLEX y genera parsers LALR. Decidí usarla porque:

- Genera el parser automáticamente a partir de la gramática
- Se integra con JFLEX (ambos usan la misma clase `sym` para los tokens)
- Produce un AST con acciones semánticas embebidas

Gramática del lenguaje (BNF):

`programa ::= instrucciones`

`instrucciones ::= instruccion instrucciones`

`| instruccion`

`instruccion ::= imprimir lparen expresion rparen scol`

`| id assign expresion scol`

`| id allocate expresion scol`

`| kwIf expresion lbrace instrucciones rbrace`

`| kwIf expresion lbrace instrucciones rbrace ELIF`

`| kwFor expresion lbrace instrucciones rbrace`

| kWBreak scol

| kWBreak

| error

expression ::= expression plus expression

| expression minus expression

| expression times expression

| expression slash expression

| expression lt expression

| minus expression

| lparen expression rparen

| kWTrue

| kWFalse

| id

| integer

| decimal

| string

Acciones semánticas:

Cada regla de producción en CUP incluye una acción semántica que construye el nodo correspondiente del AST. Por ejemplo:

```
instruccion ::= imprimir lparen expr rparen scol
```

```
{ RESULT = new Imprimir(expr); }
```

Esta acción crea un objeto `Imprimir` que contiene la expresión a mostrar.

6. Árbol de Sintaxis Abstracta (AST)

El AST es la representación jerárquica del código fuente después del análisis sintáctico. Decidí implementar el AST usando el patrón Composite combinado con el patrón Visitor por las siguientes razones:

1. Separación de preocupaciones: Los nodos solo almacenan datos, la lógica de interpretación está en el Visitor
2. Extensibilidad: Es fácil agregar nuevas operaciones (como generar código, validar tipos) sin modificar los nodos
3. Mantenibilidad: El código es más limpio y organizado

6.1 Interfaz Base ASTNode

```
public interface ASTNode {  
  
    <T> T accept(Visitor<T> visitor);  
  
}
```

Todos los nodos del AST implementan esta interfaz. El método `accept` recibe un Visitor y delega la visita al método correspondiente.

6.2 Nodos de Expresión

Clase	Representación	Ejemplo GoLite
-------	----------------	----------------

Add	Suma	<code>a + b</code>
-----	------	--------------------

Sub	Resta	<code>a - b</code>
-----	-------	--------------------

Mul	Multiplicación	<code>a * b</code>
-----	----------------	--------------------

Div	División	<code>a / b</code>
-----	----------	--------------------

Lt	Menor que	<code>a < b</code>
----	-----------	-----------------------

Negate	Negación unaria	<code>-a</code>
--------	-----------------	-----------------

Integers	Literal entero	<code>42</code>
----------	----------------	-----------------

Decimal	Literal decimal	<code>3.1416</code>
---------	-----------------	---------------------

BoolLiteral	Literal booleano	<code>true / false</code>
-------------	------------------	---------------------------

StringLiteral	Literal cadena	<code>"texto"</code>
---------------	----------------	----------------------

VarRef	Referencia a variable	x
--------	-----------------------	---

6.3 Nodos de Sentencia

Clase	Representación	Ejemplo GoLite
-------	----------------	----------------

Assign	Declaración con asignación	x := 10
--------	----------------------------	---------

Allocate	Reasignación	x = 20
----------	--------------	--------

Imprimir	Impresión	print(x)
----------	-----------	----------

IfNode	Condicional	if cond { ... }
--------	-------------	-----------------

ElseNode	Else if	else if cond { ... }
----------	---------	----------------------

ElseNodes	Lista de elifs	else if ... else if ...
-----------	----------------	-------------------------

WhileFor	Bucle for	for cond { ... }
----------	-----------	------------------

Breakstm	Break	break
----------	-------	-------

Statements

Bloque de sentencias

{ ... }

7. Patrón Visitor

El patrón Visitor es fundamental en este proyecto. Decidí implementarlo porque:

- Permite separar los algoritmos (interpretación) de la estructura de datos (AST)
- Es fácil agregar nuevas operaciones sin modificar las clases de los nodos
- Es el patrón estándar para recorrer AST en compiladores e intérpretes

7.1 Interfaz Visitor

```
public interface Visitor<T> {  
  
    T visit(Integers.Context ctx);  
  
    T visit(Decimal.Context ctx);  
  
    T visit(Add.Context ctx);  
  
    T visit(Sub.Context ctx);  
  
    T visit(Mul.Context ctx);  
  
    T visit(Div.Context ctx);  
  
    T visit(Lt.Context ctx);  
  
    T visit(Negate.Context ctx);  
  
    T visit(VarRef.Context ctx);  
  
    T visit(Assign.Context ctx);  
  
    T visit(Allocate.Context ctx);  
  
    T visit(Imprimir.Context ctx);  
  
    T visit(IfNode.Context ctx);  
}
```

```

T visit(WhileFor.Context ctx);

T visit(Breakstm.Context ctx);

T visit(Statments.Context ctx);

// ... más métodos según los nodos
}

```

7.2 Contextos Internos

Cada nodo del AST define una clase interna `Context` que encapsula los datos del nodo. Decidí usar este patrón porque:

- Permite que el nodo sea inmutable (los datos están en el Context)
- El Visitor opera sobre el Context, no sobre el nodo directamente
- Facilita la separación entre estructura y algoritmo

Ejemplo del nodo Add:

```

public class Add implements ASTNode {

    private final ASTNode left;

    private final ASTNode right;


    public static class Context {

        public final ASTNode left;

        public final ASTNode right;


        public Context(Add node) {

            this.left = node.left;

            this.right = node.right;

        }

    }

}

```



```

@Override

public <T> T accept(Visitor<T> visitor) {

    return visitor.visit(new Context(this));

}

}

```

8. Intérprete - InterpreterVisitor

El `InterpreterVisitor` es la implementación concreta del Visitor que ejecuta el código GoLite. Para esta parte, decidí implementar un intérprete recursivo que recorre el AST y evalúa cada nodo.

8.1 Estructura Principal

```

public class InterpreterVisitor implements Visitor<ValueWrapper> {

    private Enviroment environment;      // Tabla de símbolos (ámbitos)

    public List<GoliteError> errors;      // Errores semánticos

    public String output;                 // Acumulador de salida para fmt.Println

    public ValueWrapper Visit(ASTNode node) {

        return node.accept(this);

    }

    // ...

}

```

8.2 Sistema de Tipos (ValueWrapper)

Para representar los valores en tiempo de ejecución, creé una jerarquía de tipos usando `sealed interface` (característica de Java 17+). Decidí usar este enfoque porque:

- Restringe los subtipos permitidos (`IntValue`, `DecimalValue`, etc.)
- Permite usar `switch` exhaustivo en el Visitor
- Es más seguro que usar `Object` con casting

```
public sealed interface ValueWrapper
    permits IntValue, DecimalValue, VoidValue, BoolValue, StringValue {

    int line();
    int column();
    String getTypeName();
}

public record IntValue(int value, int line, int column) implements ValueWrapper {
    @Override
    public String getTypeName() { return "int"; }
}

public record DecimalValue(double value, int line, int column) implements
ValueWrapper {
    @Override
    public String getTypeName() { return "decimal"; }
}

public record BoolValue(boolean value, int line, int column) implements ValueWrapper {
    @Override
    public String getTypeName() { return "bool"; }
}

public record StringValue(String value, int line, int column) implements ValueWrapper {
    @Override
    public String getTypeName() { return "string"; }
}

public record VoidValue(int line, int column) implements ValueWrapper {
```

```

@Override
public String getTypeName() { return "void"; }
}

```

8.3 Evaluación de Expresiones

La evaluación de expresiones aritméticas soporta conversión implícita de tipos según las reglas de GoLite: cuando se opera un `int` con un `float64`, el resultado es `float64`.

Ejemplo para la suma (Add):

```

@Override
public ValueWrapper visit(Add.Context ctx) {
    ValueWrapper left = Visit(ctx.left);
    ValueWrapper right = Visit(ctx.right);

    return switch (left) {
        // int + int → int
        case IntValue l when right instanceof IntValue r ->
            new IntValue(l.value() + r.value(), l.line(), l.column());
        // int + float64 → float64
        case IntValue l when right instanceof DecimalValue r ->
            new DecimalValue(l.value() + r.value(), l.line(), l.column());
        // float64 + int → float64
        case DecimalValue l when right instanceof IntValue r ->
            new DecimalValue(l.value() + r.value(), l.line(), l.column());
        // float64 + float64 → float64
        case DecimalValue l when right instanceof DecimalValue r ->
            new DecimalValue(l.value() + r.value(), l.line(), l.column());
        default ->
            throw new RuntimeException("Operacion invalida: " + left.getTypeName() +
                " + " + right.getTypeName());
    };
}

```

8.4 Manejo de Ámbitos (Enviroment)

La clase `Enviroment` implementa la tabla de símbolos y maneja los ámbitos anidados mediante una estructura de lista enlazada (cada ámbito tiene referencia a su padre). Decidí implementarlo así porque:

- Es simple y eficiente

- Soporte natural para ámbitos anidados (bloques { })
- Fácil de depurar

```
public class Enviroment {
    private final Map<String, ValueWrapper> variables; // Ámbito actual
    private final Enviroment parent;                // Ámbito padre

    public Enviroment() {
        this.variables = new HashMap<>();
        this.parent = null; // Ámbito global
    }

    public Enviroment(Enviroment parent) {
        this.variables = new HashMap<>();
        this.parent = parent; // Ámbito anidado
    }

    // Declara una variable en el ámbito actual
    public ValueWrapper declare(String name, ValueWrapper value) {
        if (variables.containsKey(name)) {
            throw new RuntimeException("Variable " + name + " ya declarada");
        }
        variables.put(name, value);
        return value;
    }

    // Busca una variable en el ámbito actual o superiores
    public ValueWrapper get(String name) {
        if (variables.containsKey(name)) {
            return variables.get(name);
        } else if (parent != null) {
            return parent.get(name);
        }
        throw new RuntimeException("Variable " + name + " no declarada");
    }

    // Actualiza una variable existente
    public ValueWrapper set(String name, ValueWrapper value) {
        if (variables.containsKey(name)) {
            variables.put(name, value);
            return value;
        } else if (parent != null) {
            return parent.set(name, value);
        }
    }
}
```

```

        return parent.set(name, value);
    }
    throw new RuntimeException("Variable " + name + " no declarada");
}
}

```

9. Manejo de Errores

Para el manejo de errores, creé una clase unificada `GoliteError` que almacena toda la información necesaria para reportar errores al usuario. Decidí usar esta estructura porque:

- Uniformidad: Todos los errores (léxicos, sintácticos, semánticos) tienen el mismo formato
- Información completa: Tipo, descripción, línea y columna
- Fácil reporte: El método `toString()` genera el mensaje formateado

```

public class GoliteError {
    private final String type;    // "Léxico", "Sintáctico", "Semántico"
    private final String description; // Descripción del error
    private final int line;      // Línea donde ocurrió
    private final int column;    // Columna donde ocurrió

    @Override
    public String toString() {
        return String.format("Error %s: %s en la línea %d, columna %d",
                             type, description, line, column);
    }
}

```

9.1 Errores Léxicos

Se generan en `lexer.flex` cuando se encuentra un carácter no reconocido:

```

. { errors.add(new GoliteError("Lexico", "Caracter no reconocido: " + yytext(),
                               yyline, yycolumn)); }

```

9.2 Errores Sintácticos

Se generan en el parser generado por CUP cuando hay un token inesperado:

```

public void syntax_error(Symbol s) {

```

```

        errors.add(new GoliteError("Sintactico", "Simbolo inesperado: " + s.value,
                                   s.left, s.right));
    }

```

9.3 Errores Semánticos

Se generan en `InterpreterVisitor` durante la ejecución:

```

@Override
public ValueWrapper visit(VarRef.Context ctx) {
    try {
        return environment.get(ctx.name);
    } catch (RuntimeException e) {
        this.errors.add(new GoliteError("Semantico",
                                       "Variable " + ctx.name + " no declarada",
                                       ctx.line, ctx.column));
        return defaultVoid;
    }
}

```

10. Interfaz Gráfica (GUI)

Para la interfaz gráfica, utilicé Swing (biblioteca estándar de Java) junto con la librería `RSyntaxTextArea` para el editor. Decidí usar Swing porque:

- Está incluida en Java sin dependencias externas adicionales
- Es ligera y suficiente para los requerimientos del proyecto
- `RSyntaxTextArea` proporciona resaltado de sintaxis listo para usar

10.1 Clases de la GUI

Clase	Responsabilidad
<code>GoliteFrame</code>	Ventana principal, orquesta la ejecución
<code>EditorPanel</code>	Área de edición con resaltado de sintaxis
<code>GoliteMenuBar</code>	Barra de menú y botones de acción

10.2 EditorPanel

Para el editor, utilicé la librería `RSyntaxTextArea` (`com.fifesoft:rsyntaxtextarea`). Elegí esta librería porque:

- Proporciona resaltado de sintaxis para múltiples lenguajes
- Incluye plegado de código (code folding)
- Tiene auto-indentación y números de línea
- Es fácil de integrar (solo un JAR)

Configuré el editor con estilo de sintaxis Go porque GoLite está inspirado en Go:

```
textArea = new RSyntaxTextArea();
textArea.setSyntaxEditingStyle(SyntaxConstants.SYNTAX_STYLE_GO);
textArea.setCodeFoldingEnabled(true);
textArea.setAntiAliasingEnabled(true);
textArea.setAutoIndentEnabled(true);
textArea.setTabSize(4);
```

10.3 Flujo de Ejecución en la GUI

Cuando el usuario presiona el botón "Ejecutar", se ejecuta el siguiente flujo:

```
private void run() {
    try {
        // 1. Crear lexer con el texto del editor
        lexer = new Lexer(new BufferedReader(new StringReader(editorPanel.getText())));

        // 2. Crear parser con el lexer
        parser = new parser(lexer);

        // 3. Parsear para obtener el AST
        ASTNode ast = (ASTNode) parser.parse().value;

        // 4. Ejecutar el intérprete
        interpreter = new InterpreterVisitor();
        interpreter.Visit(ast);

        // 5. Mostrar la salida en consola
```

```

        cleanConsole();
        consoleTextArea.append(interpreter.output);

    } catch (Exception e) {
        consoleTextArea.append("Error: " + e.getMessage() + "\n");
    }
}

```

11. Dependencias (pom.xml)

El proyecto utiliza Maven para gestionar dependencias y automatizar la generación de analizadores. Estas son las dependencias principales:

```

<dependencies>
    <!-- Runtime para CUP (necesario para los símbolos generados) -->
    <dependency>
        <groupId>com.github.vbmacher</groupId>
        <artifactId>java-cup-runtime</artifactId>
        <version>11b-20160615-3</version>
    </dependency>

    <!-- Editor con resaltado de sintaxis -->
    <dependency>
        <groupId>com.fifesoft</groupId>
        <artifactId>rsyntaxtextarea</artifactId>
        <version>3.3.4</version>
    </dependency>
</dependencies>

```

Plugins de generación:

```

<plugins>
    <!-- Genera Lexer.java desde lexer.flex -->
    <plugin>
        <groupId>de.jflex</groupId>
        <artifactId>jflex-maven-plugin</artifactId>
        <version>1.9.1</version>
    </plugin>

    <!-- Genera parser.java y sym.java desde parser.cup -->
    <plugin>
        <groupId>com.github.vbmacher</groupId>
        <artifactId>cup-maven-plugin</artifactId>
    </plugin>

```



```
    <version>11b-20160615-3</version>
</plugin>
```

```
<!-- Agrega las fuentes generadas al classpath -->
<plugin>
  <groupId>org.codehaus.mojo</groupId>
  <artifactId>build-helper-maven-plugin</artifactId>
  <version>3.6.0</version>
</plugin>
```

```
<!-- Crea un JAR ejecutable con todas las dependencias -->
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-assembly-plugin</artifactId>
  <version>3.7.1</version>
</plugin>
</plugins>
```

12. Manual de Construcción

Para compilar y ejecutar el proyecto desde la línea de comandos:

Limpiar compilaciones anteriores

mvn clean

Generar lexer y parser, y compilar todo

mvn compile

Crear JAR ejecutable con dependencias

mvn assembly:single

Ejecutar el programa

java -jar target/clase06-1.0-jar-with-dependencies.jar

Conclusión

El intérprete GoLite implementa correctamente todas las funcionalidades requeridas para la Fase 1 del proyecto:

- Análisis léxico con JFLEX
- Análisis sintáctico con CUP
- Construcción del AST
- Interpretación mediante patrón Visitor
- Manejo de ámbitos y tabla de símbolos
- Sistema de tipos con conversión implícita `int` → `float64`
- Operadores aritméticos (+, -, *, /, %)
- Operadores relacionales (<, >, <=, >=, ==, !=)
- Operadores lógicos (&&, ||, !)
- Sentencias if-else y for
- Sentencia break
- Funciones embebidas (`fmt.Println`, `strconv.Atoi`, `strconv.ParseFloat`, `reflect.TypeOf`)
- Reporte de errores (léxicos, sintácticos, semánticos)
- Tabla de tokens
- Interfaz gráfica con editor y consola